Lesson notes · for you

Try them all, or think ahead

Optimization · Level 3 · Lesson 1

Time
20 min

Materials
Paper and a pen; Six things to pack and, if you can, a kitchen scale; Optional: a phone with Darsi to run your program

Learning goals

- Count the loads a brute-force search must try.
- Write a brute-force knapsack search in Python.
- Explain how dynamic programming is sure without trying everything.

Activity: The best bag

1. Warm-up (3 min): Choose six things for a school trip. Write each one's weight (in hundreds of grams) and points from 1 to 10, and set your bag's limit.
2. Greedy (5 min): Work out points per 100 g for each thing. Pack the highest first, then whatever still fits, and add up the points.
3. Brute force (7 min): Change w, v and the limit in your program from this lesson and run it, on a phone or on paper for a few masks. Write down the best load it finds.
4. Compare (5 min): Compare the greedy load with the best one. If they differ, find the item that took the space a better pair could have used.

No scale? Estimate: a bottle of water is about 500 g, a book about 300 g, a phone about 200 g.

Discussion

- Why does every extra item double the work for brute force?
- When is a greedy answer good enough?
- Who should choose the points in a real decision?

Check yourself

- How many loads for 10 items + one example where greedy fails.

Try it this week
Pack your bag for a school trip as a knapsack problem: pick six things, weigh them (or estimate in hundreds of grams), give each points for how much you need it, and set a weight limit. Find the best load greedily, then with your brute-force program from this lesson. Did greedy get it right?

Worksheet

Try them all, or think ahead

Optimization · Level 3 · Lesson 1

Name _____ Class _____ Date _____

1 **Your brute-force program checks all 64 loads of six items. A friend adds a seventh load. How many loads must it check now?**

◯ 65 ◯ 128 ◯ 70

2 **Taking the most points per kilo first always gives the best load.**

◯ True ◯ False

3 **Match each way of searching to what it is like.**

Brute force	• • • •	Fast, but can miss the best
Greedy rule	• • • •	You describe it; the solver searches
Dynamic programming	• • • •	Sure, but doubles with every item
A CP-SAT model	• • • •	Sure, by reusing smaller answers

4 **w and v are the weights (in 100 kg) and points of the six loads; the truck takes 10. Complete the brute-force search: for each mask, add up the weight and points of every item whose bit is 1. The program keeps the best load that fits and prints its points.**

```
w = [6, 5, 5, 3, 2, 1]
v = [42, 33, 32, 15, 8, 3]
best = 0
for mask in range(2 ** 6):
    kg = pts = 0
    # add the items whose bit is 1 in mask

    if kg <= 10 and pts > best:
        best = pts
print(best)
```

Write your program:

5 **Complete the dynamic-programming step. row[c] is the most points with room c; prev is the row before this item was considered.**

```
row = [0] * (cap + 1)
for k in range(len(w)):
    prev = row[:]
    for c in range(w[k], cap + 1):
        row[c] = _____(prev[c],
                       prev[c - _____] + _____)
```

Word bank: `v[c]` `v[k]` `w[k]` `max` `c` `min`

✎ **When have you used a quick rule, like biggest first or nearest first, that turned out not to be the best? What would checking every option have cost you?**

Check your answers

Try them all, or think ahead

Optimization · Level 3 · Lesson 1

1 **128**

Every old load can now be with or without the new item, so the count doubles: $2^7 = 128$.

2 **False**

On the 1,000 kg truck, greedy takes the solar panels first and ends with 60 points, but desks and books give 65.

3

Brute force — Sure, but doubles with every item

Greedy rule — Fast, but can miss the best

Dynamic programming — Sure, by reusing smaller answers

A CP-SAT model — You describe it; the solver searches

Brute force is honest but explodes; greedy is quick but makes no promise; dynamic programming is exact because it builds on smaller answers; and with a solver you write the rules and the goal, and it does the searching.

4

```
w = [6, 5, 5, 3, 2, 1]
v = [42, 33, 32, 15, 8, 3]
best = 0
for mask in range(2 ** 6):
    kg = pts = 0
    for i in range(6):
        if mask >> i & 1:
            kg += w[i]
            pts += v[i]
    if kg <= 10 and pts > best:
        best = pts
print(best)
```

The 64 masks are all the possible loads; the best one that fits is desks and books, 65 points, five more than greedy's 60.

5

```
row = [0] * (cap + 1)
for k in range(len(w)):
    prev = row[:]
    for c in range(w[k], cap + 1):
        row[c] = max(prev[c],
                     prev[c - w[k]] + v[k])
```

Each cell is the better (max) of skipping item k, prev[c], or taking it: its points v[k] plus the best answer for the room left after its weight, prev[c - w[k]].